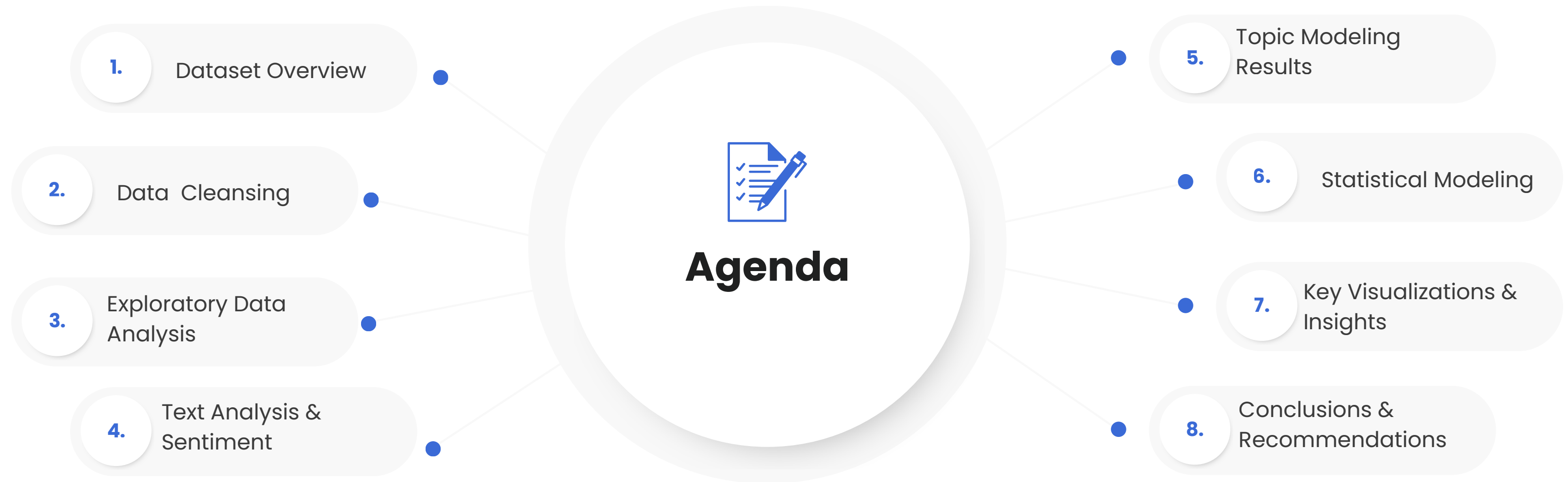


UBER | OLA

Reviews Comparison

DS804







Why This Matters?

Uber is global and standardized; Ola is built for India's unique market (traffic, payments, driver dynamics, price sensitivity). This reveals how platforms adapt to local expectations and whether riders perceive global vs. local strategies differently.



Google Play Reviews

They're authentic, unfiltered feedback from diverse riders with timestamps showing real experiences—not marketing messaging, but raw customer voice.



This Study Uniqueness

We move beyond "which is better?" to "how are these platforms adapted to India's market expectations?" and use NLP to uncover hidden themes that star ratings alone cannot reveal



Why this dataset?



Dataset Overview



Source & Size

- Total reviews: 1.7 M (balanced between platforms)
- Ola reviews: ~350K
- Uber reviews: ~1.07M
- Star ratings: 1–5 scale



Data Collection:

Google Play Store reviews from India, capturing authentic rider feedback from diverse demographics across citie.



Key Variables:

- company: Ola or Uber (categorical)
- rating: 1–5 stars (ordinal)
- review_description: raw text (unstructured)
- helpful_votes: engagement metric (thumbs-up counts)

Data Cleansing Process



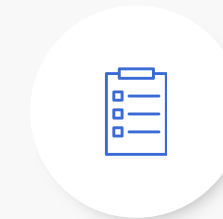
Missing Data

Identified and removed rows with null review text or missing ratings (3% of dataset).



Text Normalization

Converted to lowercase, removed extra whitespace, trimmed line breaks, eliminated special characters and punctuation for consistency.



Removed Duplicates

Ensured and changes data type for 8 columns out of 15 which is 53 %Removed reviews < 20 characters (too short, uninformative), deleted exact duplicates.

03%

Missing Data

75%

Text Normalization

05%

Removed Duplicates

Exploratory Data Analysis (EDA)

1. Overall summary by company

```
overall_summary <- reviews %>%  
  mutate(review_length = nchar(review_description)) %>%  
  group_by(company) %>%  
  summarise(  
    n_reviews    = n(),  
    avg_rating    = mean(rating, na.rm = TRUE),  
    median_rating = median(rating, na.rm = TRUE),  
    avg_thumbs_up = mean(thumbs_up, na.rm = TRUE),  
    avg_rev_length = mean(review_length, na.rm = TRUE)  
  )
```

overall_summary

company	n_reviews	avg_rating	median_rating	avg_thumbs_up	avg_rev_length
<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
Ola	357698	2.773250	2	0.8834608	92.40096
Uber	1069616	3.650441	5	0.8954054	70.23247

Exploratory Data Analysis (EDA)

2.Keyword-based counts (price, driver, cancel, safety)

```
keyword_summary <- reviews %>%
  mutate(
    text = review_description,
    price_flag = str_detect(text, "price|fare|expensive|cheap"),
    driver_flag = str_detect(text, "driver|captain|pilot"),
    cancel_flag = str_detect(text, "cancel|cancellation|cancelled"),
    safety_flag = str_detect(text, "safe|safety|unsafe|harass")
  ) %>%
  group_by(company) %>%
  summarise(
    n_reviews = n(),
    price_mentions = sum(price_flag, na.rm = TRUE),
    driver_mentions = sum(driver_flag, na.rm = TRUE),
    cancel_mentions = sum(cancel_flag, na.rm = TRUE),
    safety_mentions = sum(safety_flag, na.rm = TRUE),
    pct_price = price_mentions / n_reviews * 100,
    pct_driver = driver_mentions / n_reviews * 100,
    pct_cancel = cancel_mentions / n_reviews * 100
  )
keyword_summary
```

Exploratory Data Analysis (EDA)

3.Keyword-based counts (price, driver, cancel, safety)

Output:

company	n_reviews	price_mentions	driver_mentions	cancel_mentions	safety_mentions
<fct>	<int>	<int>	<int>	<int>	<int>
Ola	357698	22673	76863	42932	4220
Uber	1069616	52008	172888	72704	17141

Sentiment Analysis Methodology

1.Sentiment analysis (lexicon-based)

```
bing_lex <- get_sentiments("bing") # positive / negative labels
tokens_sent <- tokens %>%
  inner_join(bing_lex, by = "word") # adds 'sentiment' column
# sentiment balance per company
sentiment_company <- tokens_sent %>%
  count(company, sentiment) %>%
  group_by(company) %>%
  mutate(pct = n / sum(n) * 100)
sentiment_company
```

company	sentiment	n	pct
<fct>	<chr>	<int>	<dbl>
Ola	negative	265058	66.55033
Ola	positive	133224	33.44967
Uber	negative	461570	47.27244
Uber	positive	514834	52.72756

Sentiment Analysis Methodology

Word frequency by company (quick check)

```
word_counts_company <- tokens %>%  
  count(company, word, sort = TRUE)  
  
word_counts_company %>%  
  group_by(company) %>%  
  slice_head(n = 20)
```

company	word	n
Uber	driver	133974
Uber	service	120198
Uber	ride	114576
Uber	drivers	90887
Uber	time	82425

company	word	n
Ola	driver	58750
Ola	service	53356
Ola	worst	47272
Ola	drivers	44242
Ola	time	35300

Topic Modeling

We first converted each review into word counts, then used an LDA topic model which assumes each review is a mixture of a few hidden topics and each topic is a mixture of words. The algorithm learns which words cluster together across reviews, and outputs, for each topic, the words with highest probability. Looking at those top words, we label the topics, for example 'price issues', 'driver behaviour', 'cancellations', and 'app problems'

```
# 1) Build document-term matrix from tokens
# (each document = review_id, each term = word)
dtm_data <- tokens %>%
  count(review_id, word) %>%      # word counts per review
  filter(n > 1) %>%              # optional: drop one-off words to reduce noise
  cast_dtm(document = review_id,
           term    = word,
           value   = n)

# 2) Fit LDA topic model (choose number of topics k)
k <- 5 # you can later try 4, 6, etc.

lda_model <- LDA(dtm_data, k = k, control = list(seed = 1234))

# 3) Get top words per topic
topics_terms <- tidy(lda_model, matrix = "beta") # beta = P(word|topic)

top_terms <- topics_terms %>%
  group_by(topic) %>%
  slice_max(beta, n = 10) %>%      # top 10 words for each topic
  arrange(topic, desc(beta))

top_terms
```

Topic Modeling Results

Output:

topic	term	beta
<int>	<chr>	<dbl>
1	ride	0.28329726
1	money	0.06439441
1	hai	0.03778395
1	book	0.03107612
1	extra	0.02529286
1	charge	0.02399397
1	car	0.02375438
1	option	0.01976395
1	people	0.01344935
1	min	0.01079541
2	ola	0.22679944
2	service	0.11516961
2	customer	0.08981990
2	cab	0.08262086
2	support	0.04165956
2	auto	0.03609972
2	day	0.02788132
2	cancelled	0.02624173
2	rides	0.02419966
2	booked	0.02132452
3	app	0.26724166
3	payment	0.05148925
3	pay	0.04464715
3	rs	0.02895682
3	cash	0.02891269
3	price	0.02200883

3	card	0.02166761
3	call	0.02048144
3	paid	0.02042626
3	phone	0.01782592
4	uber	0.28789196
4	drivers	0.12671679
4	time	0.09456990
4	trip	0.06436224
4	cancel	0.06368138
4	fare	0.03256985
4	bad	0.03023152
4	charged	0.02354716
4	times	0.01472703
4	cancellation	0.01379467
5	driver	0.24624794
5	worst	0.07336399
5	location	0.04919526
5	amount	0.03969147
5	account	0.03762904
5	booking	0.03490919
5	minutes	0.02845776
5	destination	0.01693812
5	pick	0.01457557
5	route	0.01180558

Statistical Modeling Results

We fit a logistic regression to predict whether a review is positive using sentiment score, review length, and company. More positive text strongly increases the probability of a 4–5-star rating, longer reviews are more likely to be negative, and even after controlling for text and length, Uber reviews have much higher odds of being positive than Ola reviews. All effects are highly statistically significant.

```
Call:
glm(formula = positive_review ~ sentiment_score + review_length +
     company, family = binomial, data = reviews_model)

Coefficients:
              Estimate Std. Error  z value Pr(>|z|)
(Intercept)  -4.704e-02  7.915e-03   -5.943 2.79e-09 ***
sentiment_score  1.532e+00  3.591e-03  426.696  < 2e-16 ***
review_length  -1.155e-02  4.606e-05 -250.696  < 2e-16 ***
companyUber     9.947e-01  8.600e-03  115.667  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 1089716  on 786382  degrees of freedom
Residual deviance:  474362  on 786379  degrees of freedom
AIC: 474370

Number of Fisher Scoring iterations: 7
```


Statistical Modeling Methodology

```
word_counts_company <- tokens %>%
  count(company, word, sort = TRUE)

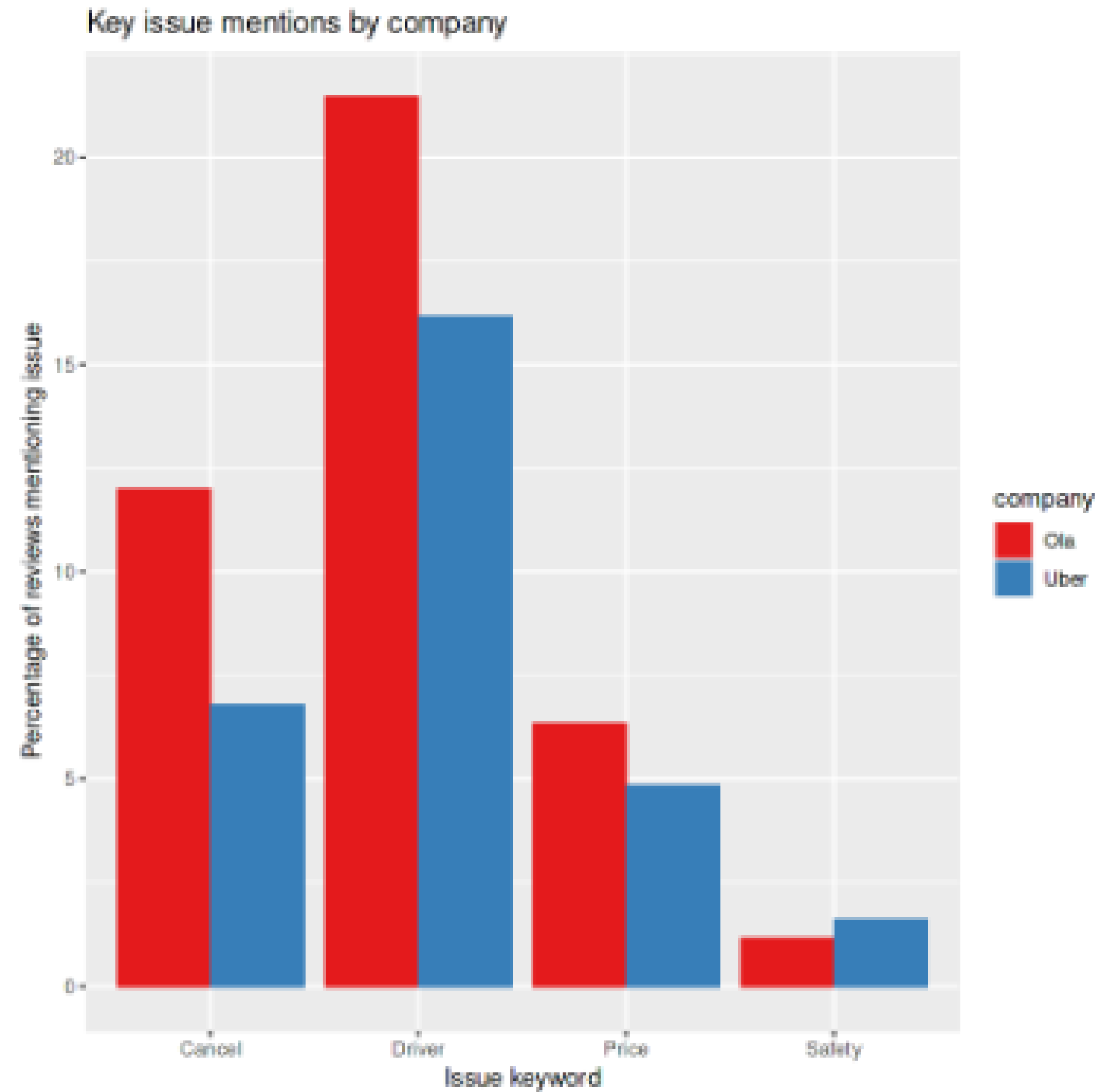
word_counts_company %>%
  group_by(company) %>% # 1. Create binary target and merge sentiment with main data
reviews_model <- reviews %>%
  mutate(
    review_length = nchar(review_description),
    positive_review = rating >= 4
  ) %>%
  left_join(review_sentiment %>% select(review_id, sentiment_score),
    by = "review_id") %>%
  mutate(
    positive_review = as.integer(positive_review) # 1 = positive, 0 = not
  ) %>%
  select(positive_review, sentiment_score, review_length, company) %>%
  drop_na()

# 2. Fit logistic regression
logit_model <- glm(
  positive_review ~ sentiment_score + review_length + company,
  data = reviews_model,
  family = binomial
)

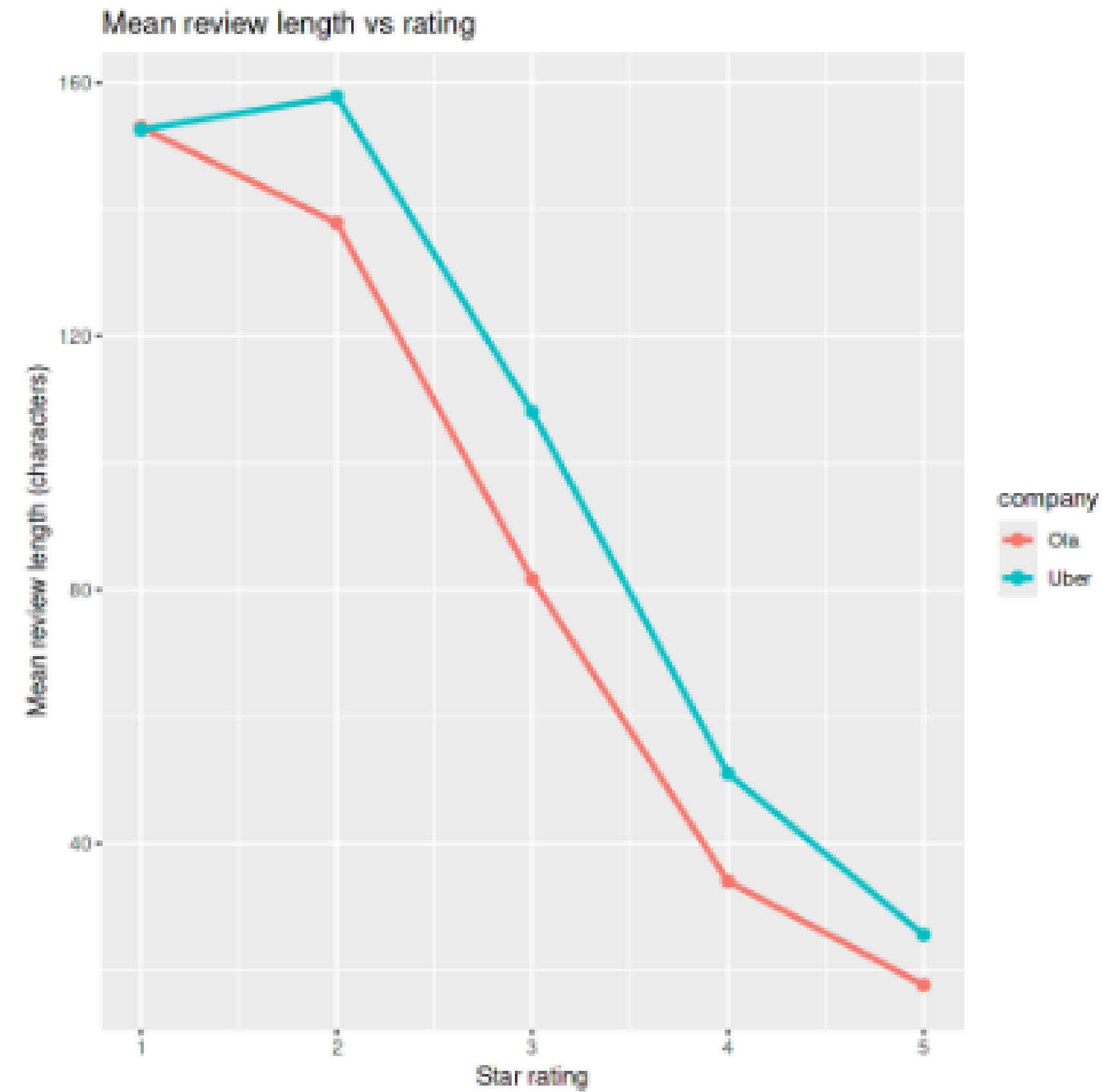
summary(logit_model)

slice_head(n = 20)
```

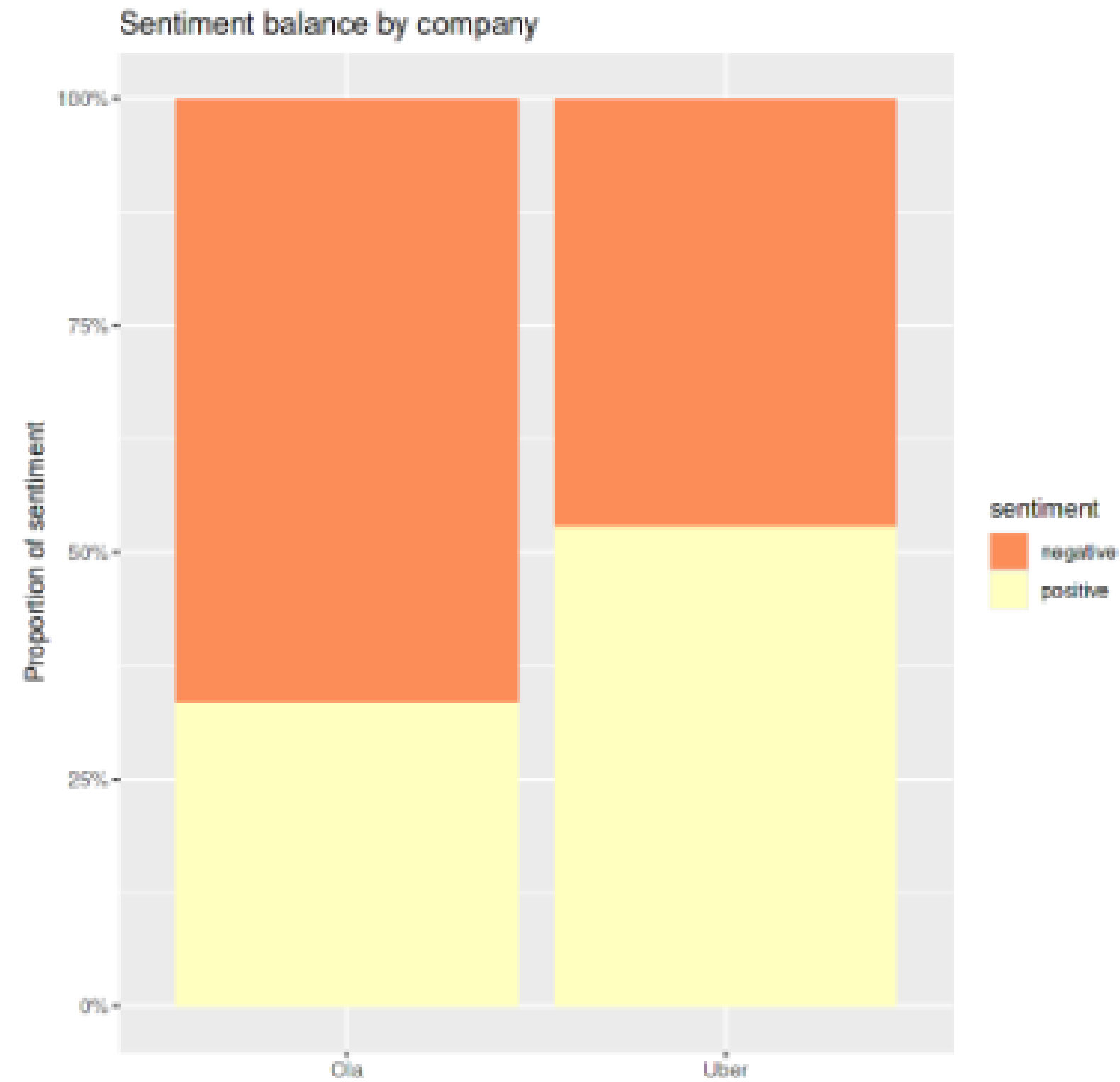
Visulization 1



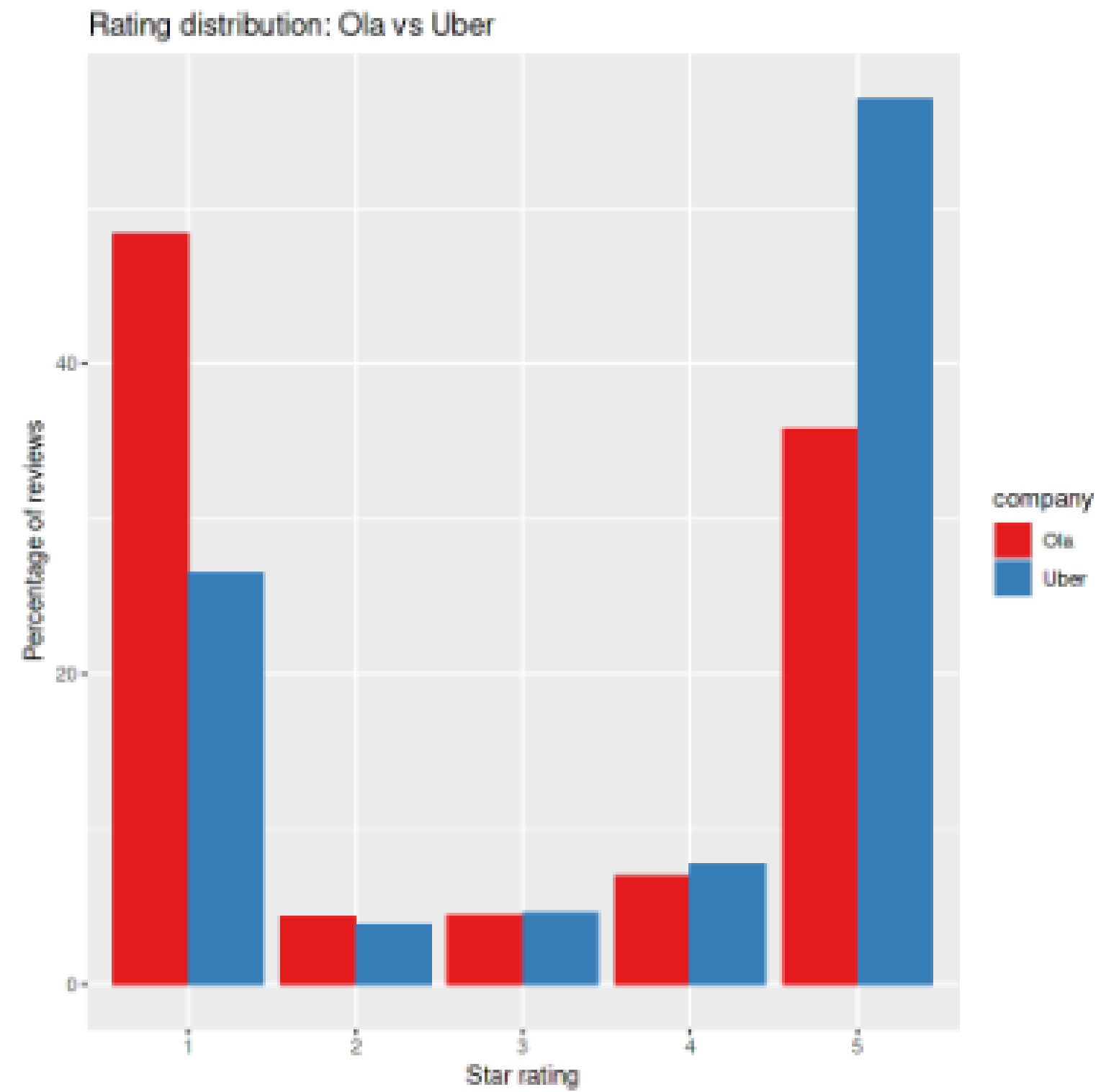
Visualization 2



Visualization 3



Visualization 4



Conclusion

- Ola reviewers write longer reviews
- OLA has a higher percentage of negative reviews
- Uber reviews have a higher probability of being positive
- “Driver” was highly mentioned among both in negative reviews
- Bad reviews tend to be longer than good reviews
- Uber has a significantly higher percentage of 5 star ratings and OLA has a significantly higher percentage of 1 star ratings



**Thank
You**

